Tutoriel : Support Multi-Profils pour un Utilisateur

Ce tutoriel explique comment transformer le système actuel (monoprofil) pour permettre à un utilisateur d'assumer plusieurs rôles à la fois (par exemple, être à la fois **Acheteur** et **Chef de Projet**).

🏗️ Architecture du changement :

Nous allons passer d'un champ `profileId: String` à `profiles: String[]` (tableau de chaînes) pour stocker tous les rôles de l'utilisateur.

Étape 1 : Modification du Schéma (Backend)

Ouvrez `server/prisma/schema.prisma` et mettez à jour le modèle `User`.

```
model User {
  id          String    @id @default(uuid())
  email       String    @unique
  // ...
  // profileId String   <-- À REMPLACER
  roles       String[]  @default(["VIEWER"]) // Système de rôles multiples
}
```

Appliquez la migration :

```
npx prisma migrate dev --name enable_multi_roles
```

Étape 2 : Signature du Token JWT (Auth)

Lors de la connexion (`server/src/routes/auth.js`), modifiez le payload du token pour inclure tous les rôles.

```
const token = jwt.sign(
  {
    id: user.id,
    email: user.email,
    roles: user.roles // On envoie maintenant le tableau complet
  },
  process.env.JWT_SECRET
);
```

Étape 3 : Mise à jour du Middleware (Sécurité)

Si vous avez un middleware qui vérifie les rôles, il doit maintenant vérifier si *l'un* des rôles est autorisé.

```
// Exemple de fonction utilitaire de vérification
function hasRole(user, requiredRole) {
  return user.roles && user.roles.includes(requiredRole);
}

// Dans une route :
if (!hasRole(req.user, 'ADMIN') && !hasRole(req.user, 'RPROG')) {
  return res.status(403).json({ error: "Accès refusé" });
}
```

Étape 4 : Gestion du Frontend (AuthContext)

Mettez à jour `src/contexts/AuthContext.jsx` pour exposer des méthodes de vérification simplifiées.

```
// AuthContext.jsx
const hasRole = (role) => user?.roles?.includes(role);

// Utilisation dans un composant :
const { hasRole } = useAuth();

return (
  <>
    {hasRole('ADMIN') && <AdminTools />}
    {(hasRole('CHEF_PROJET') || hasRole('RPROG')) && <ProjectEditButton />}
  </>
);
```

Étape 5 : Migration des Données

Étant donné que vous passez d'un champ simple à un tableau, créez un script SQL ou une fonction Prisma pour migrer les anciens `profileId` vers le nouveau tableau `roles`.

```
// Script de migration rapide
const users = await prisma.user.findMany();
for (const user of users) {
  await prisma.user.update({
    where: { id: user.id },
    data: { roles: [user.profileId] } // Enveloppe l'ancien rôle dans un tableau
  });
}
```

✅ Points de vigilance :

1. **Priorité des rôles** : Si un utilisateur est à la fois `ADMIN` et `VIEWER`, c'est toujours le rôle le plus puissant (`ADMIN`) qui doit l'emporter dans votre logique métier.
2. **Interface de gestion** : La page de gestion des utilisateurs doit maintenant proposer une liste de cases à cocher (Checkboxes) au lieu d'un menu déroulant (Select).

> [!TIP] Dans une architecture complexe, il est souvent préférable de garder un rôle "Principal" et d'y ajouter des "Permissions" granulaires. L'approche multi-profils ci-dessus est la plus simple pour démarrer.